

Function

A **function in Python** is a **block of reusable code** that performs a specific task. You write it once and can **call (use) it many times** in your program.

Type of function:-

1. In-built function
2. User define function

In-built function:-

1. print()
2. type()
3. id()
4. max()
5. min()
6. len()
7. sum()
8. eval() etc.....

User define function:-

Syntax:

```
def func_name(parameters....):  
    fun_body.....  
    fun_body.....  
    fun_body.....  
    return .....  
  
result = func_name(arguments....)
```

***required :-** def, func_name, () , :

optional :- parameters , return , arguments

Type of user define function:-

1. Function with no arguments & no return value

```
def func_name():  
    fun_body.....  
    fun_body.....  
    fun_body.....  
  
result = func_name()
```

2. Function with arguments & no return value

```
def func_name(parameters....):  
    fun_body.....  
    fun_body.....  
    fun_body.....  
  
result = func_name(arguments....)
```

3. Function with no arguments & with return value

```
def func_name():  
    fun_body.....  
    fun_body.....  
    fun_body.....  
    return .....  
  
result = func_name()
```

4. Function with arguments & with return value

```
def func_name(parameters....):  
    fun_body.....  
    fun_body.....  
    fun_body.....  
    return .....  
  
result = func_name(arguments....)
```

Type of function arguments:

1. Positional argument.
2. Default positional argument.
3. Variable length positional argument.
4. Keyword argument.
5. Default keyword argument.
6. Variable length keyword argument.

1. Positional argument:-

Syntax:

```
def fun_name(param1, param2, param3):  
    function body  
    function body  
    function body
```

```

fun_name(arg1, arg2, arg3)
fun_name() # error
fun_name(arg1) # error
fun_name(arg1, arg2) # error

```

2. Default positional argument:-

Syntax:

```

def fun_name(param1=0, param2=0, param3=0):
    function body
    function body
    function body
fun_name()
fun_name(arg1)
fun_name(arg1, arg2)
fun_name(arg1, arg2, arg3)
fun_name(arg1, arg2, arg3, arg4) # error

```

3. Variable length positional argument:-

Syntax:

```

def fun_name(*args):
    print(args)
    print(type(args))
    function body
fun_name()
fun_name(arg1)
fun_name(arg1, arg2)
fun_name(arg1, arg2, arg3)
fun_name(arg1, arg2, arg3, arg4)

```

4. Keyword argument:-

Syntax:

```

def fun_name(param1, param2, param3):
    function body
    function body
    function body
fun_name(param1=arg1, param2=arg3, param3=arg2)
fun_name()
# error
fun_name(param1=arg1) # error
fun_name(param1=arg1, param2=arg3) # error

```

5. Default keyword argument:-

Syntax:

```
def fun_name(param1=0, param2=0, param3=0):  
    function body  
    function body  
    function body  
fun_name()  
fun_name(param1=arg1)  
fun_name(param1=arg1,param2=arg3)  
fun_name(param1=arg1,param2=arg3,param3=arg2)  
fun_name(param1=arg1,param2=arg3,param3=arg2, param4=arg4)    #error
```

6. Variable length keyword argument.:-

Syntax:

```
def fun_name(**kwargs):  
    print(kwargs)                # {'key': 'value'}  
    print(type(kwargs))          # <class 'dict'>  
    function body  
fun_name()  
fun_name(param1=arg1)  
fun_name(param1=arg1,param2=arg3)  
fun_name(param1=arg1,param2=arg3,param3=arg2)  
fun_name(param1=arg1,param2=arg3,param3=arg2, param4=arg4)
```